PLUGIN_GUIDE

Challenges Module - Plugin Guide

Overview

The plugin system allows extending the Challenges framework with custom challenge types, assertion evaluators, and integrations without modifying the core module.

Plugin Interface

```
type Plugin interface {
    // Name returns the plugin name (must be unique).
    Name() string

    // Version returns the plugin version (semver recommended).
    Version() string

    // RegisterChallenges registers challenge implementations.
    RegisterChallenges(reg registry.Registry) error

    // RegisterAssertions registers custom assertion evaluators.
    RegisterAssertions(engine assertion.Engine) error
}
```

Creating a Plugin

Step 1: Define Your Plugin

```
package myplugin

import (
    "digital.vasic.challenges/pkg/assertion"
    "digital.vasic.challenges/pkg/registry"
```

```
)

type MyPlugin struct {
    config Config
}

type Config struct {
    APIBaseURL string
    Timeout    time.Duration
}

func New(cfg Config) *MyPlugin {
    return &MyPlugin{config: cfg}
}

func (p *MyPlugin) Name() string    { return "my-plugin" }
func (p *MyPlugin) Version() string { return "1.0.0" }
```

Step 2: Register Challenges

```
func (p *MyPlugin) RegisterChallenges(
    reg registry.Registry,
) error {
    // Register your custom challenges
    if err := reg.Register(NewEndpointChallenge(p.config)); err !
        = nil {
        return err
    }
    if err := reg.Register(NewLatencyChallenge(p.config)); err !=
        nil {
        return err
    }
    return nil
}
```

Step 3: Register Custom Assertions

```
func (p *MyPlugin) RegisterAssertions(
    engine assertion.Engine,
) error {
    // Register domain-specific assertions
    engine.Register("api_status_ok", func(
        def assertion.Definition, value any,
    ) (bool, string) {
        code, ok := value.(int)
        if !ok {
            return false, "value is not an int"
        }
        if code >= 200 && code < 300 {
            return true, fmt.Sprintf("status %d is OK", code)
        }
```

```
        return false, fmt.Sprintf("status %d is not OK", code)
    })

    return nil
}
```

Step 4: Load the Plugin

```
import "digital.vasic.challenges/pkg/plugin"

pluginReg := plugin.NewPluginRegistry()

// Register plugins
pluginReg.Register(myplugin.New(myplugin.Config{
    APIBaseURL: "http://localhost:8080",
    Timeout:    30 * time.Second,
}))

// Load all plugins into the challenge system
err := pluginReg.LoadAll(challengeRegistry, assertionEngine)
```

Best Practices

1. **Unique names**: Each plugin must have a unique name
2. **Versioning**: Use semantic versioning for plugins
3. **Error handling**: Return meaningful errors from registration
4. **Dependencies**: Declare challenge dependencies correctly
5. **Idempotency**: Registration should be safe to call multiple times
6. **Cleanup**: Implement `Cleanup()` on challenges to release resources
7. **Context**: Always respect context cancellation in Execute()
8. **Testing**: Test plugins independently before integration